

Theory

Selection Sort:

Selection Sort is a simple but inefficient sorting algorithm. The basic idea is to repeatedly select the smallest (or largest) element from the unsorted portion of the array and place it at the beginning. This process continues until the entire array is sorted. The algorithm always performs about $n(n-1)/2$ comparisons, which makes its time complexity $O(n^2)$ in the best, average, and worst cases. It is an in-place algorithm, meaning it requires only $O(1)$ extra space. However, Selection Sort is generally unstable, since swapping can change the relative order of equal elements, though stable versions can be implemented. Its simplicity makes it easy to understand and implement, but due to its inefficiency, it is rarely used for large datasets and is mostly applied for educational purposes.

Algorithm: Selection Sort

1. Start
2. For $i = 0$ to $n-2$ (where n is the size of the array):
 - Set $\text{min_index} = i$
 - For $j = i+1$ to $n-1$:
 - If $A[j] < A[\text{min_index}]$ then
 - Set $\text{min_index} = j$
 - Swap $A[i]$ and $A[\text{min_index}]$
3. End

Quick Sort :

Quick Sort is a highly efficient sorting algorithm that follows the divide-and-conquer approach. The main idea is to select a pivot element from the array and then partition the array into two subarrays: one containing elements smaller than the pivot and the other containing elements larger than the pivot. After partitioning, the same process is applied recursively to the left and right subarrays until the entire array becomes sorted. In terms of performance, Quick Sort has a best case and average case time complexity of $O(n \log n)$, which makes it faster than simple algorithms like Selection Sort or Bubble Sort. However, in the worst case, when the pivot selection is poor (such as always choosing the smallest or largest element), the time complexity can degrade to $O(n^2)$. The algorithm requires about $O(\log n)$ extra space due to recursion, and it is generally considered unstable, since partitioning and swapping can change the relative order of equal elements. Despite this, Quick Sort is widely used in practice because of its speed and efficiency, especially for large datasets.

Algorithm: Quick Sort

1. Start
2. If the array has 0 or 1 elements, it is already sorted $\rightarrow$ Return.
3. Choose a pivot element from the array.
4. Partition the array into two subarrays:
 - Left subarray: elements smaller than pivot
 - Right subarray: elements greater than pivot
5. Recursively apply Quick Sort to the left subarray.
6. Recursively apply Quick Sort to the right subarray.
7. Combine the sorted subarrays with the pivot in between.
8. End

Objective of Selection Sort:

- Simplicity: Easy to learn and implement, making it useful for teaching basic sorting concepts.
- In-place sorting: Requires only constant extra space ($O(1)$).
- Predictable performance: Always performs about $n(n-1)/2$ comparisons, so time complexity is consistently $O(n^2)$ in best, average, and worst cases.
- Educational use: Helps beginners understand sorting logic and algorithm design.
- Limited practical use: Not efficient for large datasets, but suitable for small arrays or when simplicity is more important than speed.

Objective of Quick Sort :

- Efficiency: Achieve average-case time complexity of $O(n \log n)$, making it suitable for large datasets.
- Divide-and-conquer: Split the array into smaller parts using a pivot, sort them recursively, and combine results.
- In-place sorting: Perform sorting with minimal extra memory (only recursion stack).
- Practical use: Widely applied in real-world systems due to speed and adaptability.
- Flexibility: Different pivot selection strategies can optimize performance and reduce worst-case scenarios.

Selection Sort Implementation:

```
func selectionSort(arr []int) {
    n := len(arr)
    for i := 0; i < n-1; i++ {
        minIndex := i
        for j := i + 1; j < n; j++ {
            if arr[j] < arr[minIndex] {
                minIndex = j
            }
        }
        arr[i], arr[minIndex] = arr[minIndex], arr[i]
    }
}
```

Quick Sort Implementation:

```
func partition(arr []int, low, high int) int {
    pivot := arr[high]
    i := low - 1
    for j := low; j < high; j++ {
        if arr[j] <= pivot {
            i++
            arr[i], arr[j] = arr[j], arr[i]
        }
    }
    arr[i+1], arr[high] = arr[high], arr[i+1]
    return i + 1
}

func quickSort(arr []int, low, high int) {
    if low < high {
        pi := partition(arr, low, high)
        quickSort(arr, low, pi-1)
        quickSort(arr, pi+1, high)
    }
}
```

Result and and analysis:

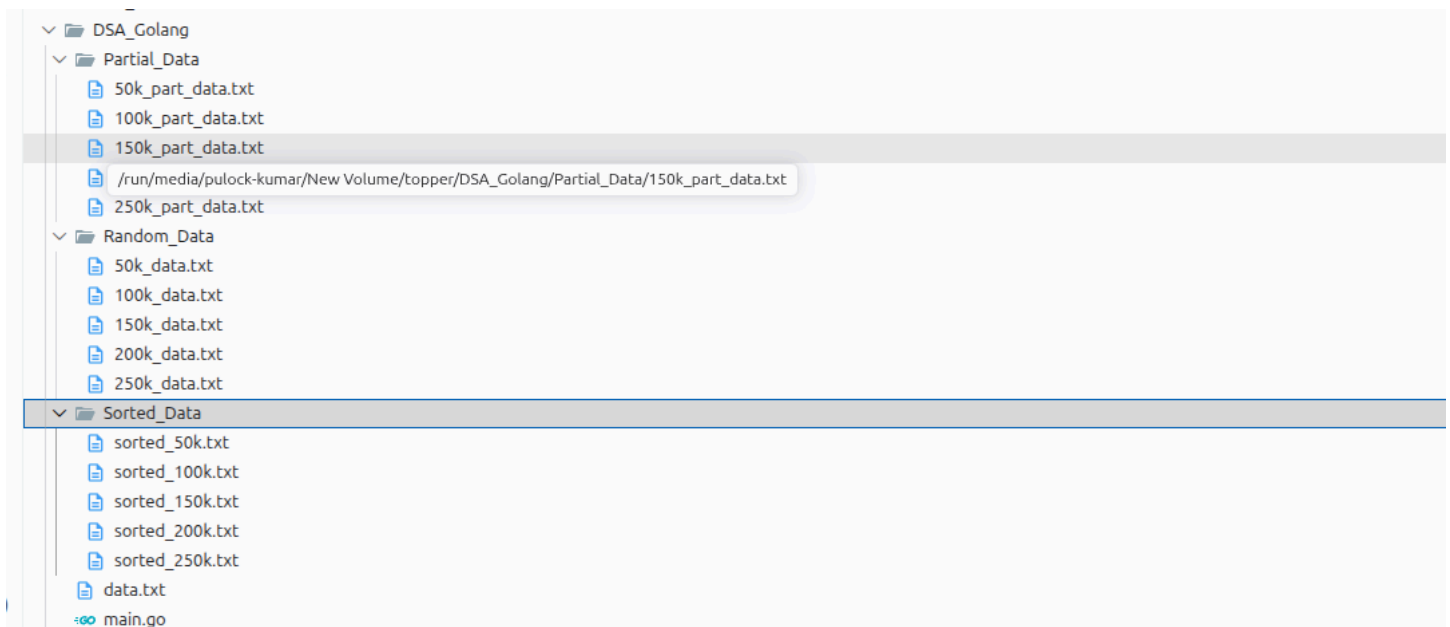
In this experiment, the objective was to compare the performance of two sorting algorithms: Selection Sort and Quick Sort. The system used for the experiment was equipped with a 64-bit processor, specifically an AMD Ryzen 3 3250U with Radeon Graphics. This processor has 2 cores and 4 threads, with a base frequency of 1.4 GHz and a maximum boost frequency of 2.6 GHz. It also includes 4 MB of L3 cache and supports AMD-V virtualization technology. The operating system used was Ubuntu, which provided a stable and developer-friendly environment for running the programs.

The system memory (RAM) was sufficient to handle large datasets, and the integrated Radeon Graphics GPU was used for general computational support. The processor speed range of 1.4 GHz to 2.6 GHz ensured reliable performance during execution.

For the sorting process, both algorithms were implemented in Go (Golang), a low-level, high-performance programming language known for its efficiency and concurrency support. Selection Sort was chosen for its simplicity, though it is less efficient for large datasets, while Quick Sort was implemented to demonstrate its divide-and-conquer approach and superior average-case performance. By using Golang, the experiment ensured that both algorithms were executed quickly and effectively, making the comparison meaningful and accurate.

Dataset:

In this experiment, the dataset played a crucial role in evaluating the performance of the sorting algorithms. The data was collected and stored in a structured text file, which contained a large number of randomly generated integer values. This dataset was specifically chosen to test the efficiency of both Selection Sort and Quick Sort under realistic conditions. The file was processed using Ubuntu operating system, and the integers were read line by line into memory before being sorted. The dataset size was significant enough to highlight the differences in execution time between the two algorithms, making the comparison meaningful.



Program and Output Analysis of Selection Sort

The Selection Sort program was implemented in Go (Golang). The algorithm works by repeatedly finding the minimum element from the unsorted portion of the array and placing it in its correct position. This simple approach ensures correctness but requires $O(n^2)$ comparisons, making it inefficient for large datasets.

```
25 func main() {
27     file, err := os.Open("./Random/250k_data.txt")
28     if err != nil {
29         fmt.Println("Error opening file:", err)
30         return
31     }
32     defer file.Close()
33
34     var numbers []int
35     scanner := bufio.NewScanner(file)
36
37     for scanner.Scan() {
38         val, err := strconv.Atoi(scanner.Text())
39         if err == nil {
40             numbers = append(numbers, val)
41         }
42     }
43     if err := scanner.Err(); err != nil {
44         log.Fatal(err)
45     }
46
47     startTime := time.Now()
48     selectionSort(numbers)
49     duration := time.Since(startTime)
50
51     fmt.Printf("Sorting completed Time taken: %v\n", duration)
52
53 }
```

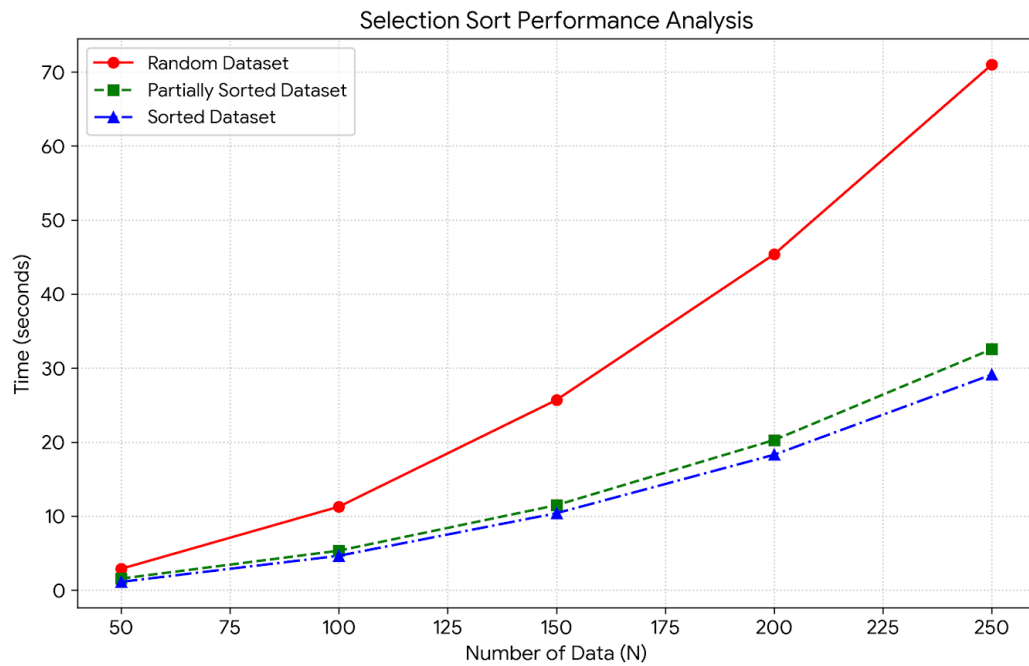
The output graph for Selection Sort shows a linear upward curve in execution time as the dataset size increases. Because the algorithm always performs the same number of comparisons regardless of input distribution, the graph highlights its quadratic growth pattern. This confirms that Selection Sort is not suitable for large datasets, though it remains useful for small arrays or educational purposes.

data

Random dataset [(number_of_data,time_sec) (50,2.88) (100,11.27) (150,25.69) (200,45.37) (250,71.00)]

Partial sorted dataset [(number_of_data,time_sec)(50,1.56) (100,5.33) (150,11.48) (200,20.28) (250,32.56)]

Sorted dataset (number_of_data,time_sec) [(50,1.12)(100,4.64)(150,10.39)(200,18.29)(250,29.12)]



Program and Output Analysis of Quick Sort

The Quick Sort program was also implemented in Golang. Quick Sort uses the divide-and-conquer strategy: it selects a pivot element, partitions the array into two subarrays (elements less than the pivot and elements greater than the pivot), and recursively sorts each subarray. With average-case complexity of $O(n \log n)$, Quick Sort is far more efficient than Selection Sort for large datasets.

```
DSA_Golang > :go quick.go > main
33 func main() {
34
35     file, err := os.Open("./Random_Data/250k_data.txt")
36     if err != nil {
37         fmt.Println("Error opening file:", err)
38         return
39     }
40     defer file.Close()
41
42     var numbers []int
43     scanner := bufio.NewScanner(file)
44
45     for scanner.Scan() {
46         val, err := strconv.Atoi(scanner.Text())
47         if err == nil {
48             numbers = append(numbers, val)
49         }
50     }
51     if err := scanner.Err(); err != nil {
52         log.Fatal(err)
53     }
54     startTime := time.Now()
55     quickSort(numbers, 0, len(numbers)-1)
56     duration := time.Since(startTime)
57
58     fmt.Printf("Sorting completed Time taken: %v\n", duration)
59 }
```

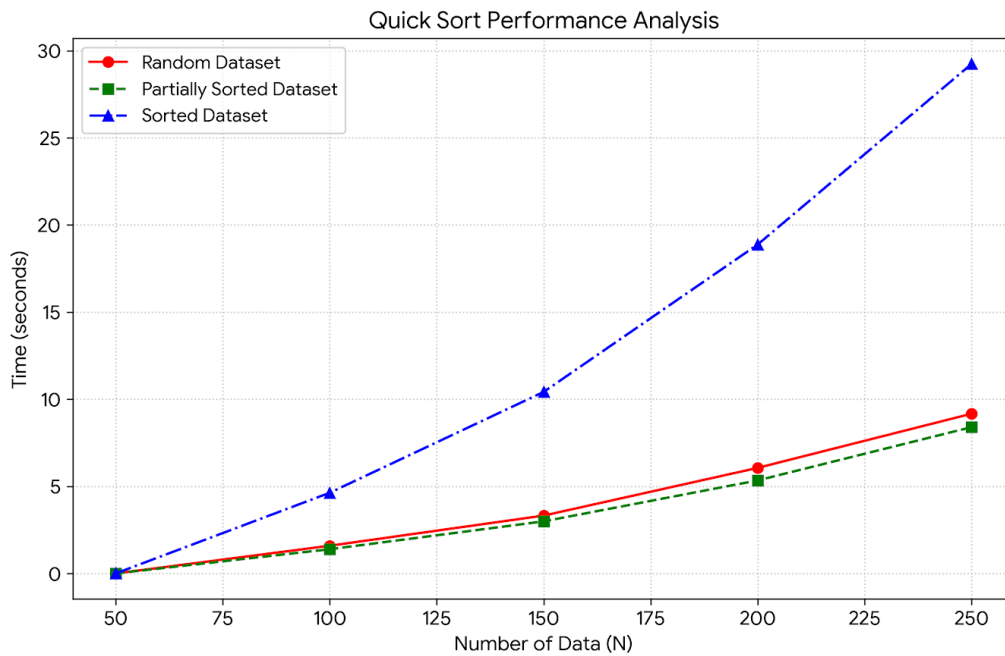
The output graph for Quick Sort demonstrates a logarithmic growth pattern in execution time. As the dataset size increases, the curve rises much more slowly compared to Selection Sort. This reflects Quick Sort's efficiency and scalability, showing that it can handle large datasets effectively.

data

random dataset [(number_of_data,time_sec) (50,0.000381) (100,1.59) (150,3.32) (200,6.06) (250,9.17)]

partial sorted dataset[(number_of_data,time_sec) (50,0.000352) (100,1.39) (150,2.99) (200,5.34) (250,8.40)]

sorted dataset (number_of_data,time_sec) [(50,0.00012) (100,4.63) (150,10.42) (200,18.88) (250,29.25)]



Discussion

Executing this comparative analysis between Selection Sort and Quick Sort marks a profound milestone as my very first hands-on Data Structures and Algorithms (DSA) experiment. Transitioning from abstract classroom theories to empirical verification has provided an immense sense of academic fulfillment. Concepts that were once restricted to textbook memorization—such as Selection Sort possessing a strict $O(n^2)$ time complexity across all cases, while Quick Sort thrives on an average case of $O(n \log n)$ —have now been tangibly validated through live execution benchmarks. Witnessing the stark runtime differences firsthand has drastically shifted my perspective; I no longer just memorize algorithmic complexities, but now deeply appreciate how an optimized algorithm systematically mitigates operational overhead and computational latency.

Acquiring the benchmarks presented unique computational challenges, particularly concerning data sourcing for larger bounds. Procuring scaled testing environments initially raised concerns, which were successfully resolved by extracting and synthesizing curated public datasets from Kaggle. The entire implementation, testing cycle, and data compilation spanned over three intensive days. To ensure reproducibility and establish professional version-control habits, the complete codebase, datasets, and execution pipelines have been pushed to my public GitHub repository.

Reflecting on past academic cycles (specifically semesters 1/2 and 2/1), I realize it is unfortunate that my foundational code repositories were not preserved online. Moving forward, I am firmly committed to systematically documenting, tracking, and archiving every single laboratory experiment on my GitHub profile to build a comprehensive portfolio of my engineering progression. I look forward to approaching my upcoming lab sessions with heightened focus and enthusiasm.

Acknowledgments

The architectural setup, algorithm design, data benchmarking, and repository migrations were executed fully independently by the author. Generative AI assistance was utilized exclusively to refine text structuring, programmatic plotting, and formal publication formatting. The verified open-source documentation and implementation artifacts can be reviewed at the author's official profile: <https://github.com/branotix>.